

GPTHub Prod — архитектура и User Flow

GPTHub Prod — краткая архитектура и User Flow (сопроводительный текст)

=====

Для загрузки в форму используйте один файл: GPTHub_architecture_submission.pdf (текст этой страницы + две схемы). Исходники схем: architecture.png, user_flow.png (из Mermaid .mmd). Для автоматической проверки ИИ см. UTF-8 Markdown ARCHITECTURE_FOR_AI.md (тот же смысл + Mermaid в тексте + пересказ схем).

Продукт: один чат в Open WebUI; сценарии (текст, файлы, картинка, аудио, URL, голос через STT/TTS UI, веб-поиск при включении, WOW: совет моделей, WOW: PPTX) идут в один OpenAI-совместимый запрос к orchestrator, без второго флагманского режима.

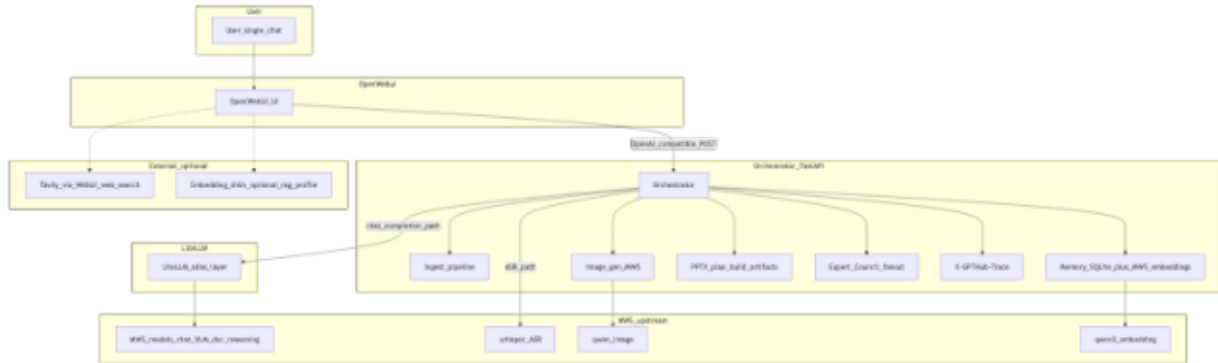
Контур сервисов: Open WebUI → orchestrator (FastAPI: ingest, classifier/router, memory, council, image-gen, PPTX, trace) → LiteLLM (алиасы) → MWS (upstream чат/VLM/doc/reasoning).
Наблюдаемость: заголовок X-GPTHub-Trace.

Модели (см. infra/litellm/config.yaml, docs/MWS_CATALOG.md): baseline mws-gpt-alpha (gpt-hub-turbo), тяжёлый glm-4.6-357b (gpt-hub-strong), документный qwen2.5-72b-instruct (gpt-hub-doc), код/рассуждения qwen3-coder-480b-a35b (gpt-hub-reasoning-or), VLM-цепочка qwen3-vl-* / qwen2.5-vl* / cotypr-pro-vl-32b, fallback gemma-3-27b-it; image-gen — qwen-image (MWS /v1/images/generations); ASR — whisper-medium; память — qwen3-embedding-8b; план PPTX — алиасы gpt-hub-pptx-* и др.

Внешние зависимости: MWS (API, .env / .env.mws.local); Docker Compose; SQLite для фактов памяти; опционально Tavily (WebUI web search, ENABLE_WEB_SEARCH, TAVILY_API_KEY, bypass эмбединга сниппетов); markdown для DOCX/XLSX/PPTX ingest; опционально embedding shim (профиль rag). Без MWS основной чат не работает.

Исходники диаграмм (Mermaid): docs/submission/architecture.mmd, docs/submission/user_flow.mmd.
Рендер: mmdc (пакет @mermaid-js/mermaid-cli).

Контур сервисов и моделей



User Flow — один чат

